



Tecnológico de Monterrey - Campus Monterrey
Programación de estructuras de datos y algoritmos fundamentales

**Reflexión de: Evidencia 1 - Actividad Integral de Conceptos Básicos y
Algoritmos Fundamentales**

Juan Ángel Narváez Torres - A00843809

Septiembre 20, 2026
Monterrey, Nuevo León

A. Importancia de ordenar y buscar en este tipo de logs

En la vida real, ordenar y poder buscar en los archivos de bitácoras (logs) es muy importante. Si ocurre un error en el sistema o un ataque, los logs son nuestra única forma de saber qué pasó y a qué hora. Pero como hay varios servidores escribiendo al mismo tiempo, casi siempre hay un pequeño desfase en los tiempos de los registros. Ordenar todo cronológicamente nos permite ver la historia real de los eventos en orden. Por otro lado, la búsqueda por rango nos ayuda a filtrar y encontrar errores en ventanas de tiempo específicas sin tener que revisar millones de líneas a mano, lo cual sería imposible.

B. Comparación conceptual de los algoritmos elegidos

Para esta actividad metimos los algoritmos que vimos en clase: Swap, Bubble, Insertion, Selection, Quick y Merge Sort.

- **Merge Sort y Quick Sort:** Son los mejores para el archivo que está totalmente desordenado porque son mucho más eficientes (su complejidad es $O(n \log n)$)
- **Insertion Sort:** Lo interesante pasa con este algoritmo en el archivo casi ordenado. Aunque su peor caso es $O(n^2)$, está diseñado de forma que, si los datos ya están casi en su posición correcta, el ciclo while interno se rompe muy rápido. Esto hace que en la práctica termine comportándose casi como una complejidad lineal $O(n)$. Otros algoritmos como Bubble o Selection Sort no aprovechan esto y siempre hacen comparaciones de más.

C. Interpretación de mediciones contrastadas con la teoría

Antes de correr las pruebas, mi predicción era que Merge Sort siempre iba a ser el más rápido porque es el más eficiente en la teoría. Pero al probar el archivo casi ordenado, me di cuenta de que Insertion Sort le ganó por mucho. Esto tiene sentido porque Merge Sort sigue dividiendo y juntando los arreglos completos sin importar si ya están ordenados, mientras que Insertion Sort aprovecha el orden que ya traen los datos. En el archivo desordenado, ahí sí Merge Sort y Quick Sort demostraron ser muchísimo más rápidos que los demás.

D. Uso de Inteligencia Artificial

Usé Gemini como apoyo durante la programación para resolver dudas específicas con estos 3 prompts clave:

1. **Prompt 1:** "¿Cómo puedo convertir una fecha tipo 'Sep 12 2024 05:00:00' a un número en C++ para poder ordenarlo más fácil con los algoritmos que ya tengo?". Me recomendó usar la librería `<sstream>` y pasar todo a un tipo `long long` multiplicando por factores (año, mes, día, etc.) para crear una sola llave numérica. Lo adapté y lo metí en mi método `calcularFechaValor()`.
2. **Prompt 2:** "¿Por qué Insertion Sort corre más rápido que Merge Sort cuando le paso un arreglo que ya está casi ordenado?". Me sirvió para confirmar la parte teórica de que su complejidad baja a $O(n)$ en el mejor de los casos.
3. **Prompt 3:** "Genera una función de búsqueda binaria en C++ para encontrar una fecha de inicio y fin en un vector de objetos ordenados."
 - **El error de la IA:** Me dio el código clásico de la búsqueda binaria que retorna el índice en el momento exacto en que encuentra el valor.
 - **Cómo lo detecté:** Al hacer las pruebas con el archivo `log607-2.txt`, que trae fechas duplicadas a propósito, vi que el programa no me imprimía todos los registros del rango. El algoritmo de la IA se detenía en medio de los duplicados y dejaba fuera a los demás.
 - **Cómo lo corregí:** No usé el código de la IA. Para arreglarlo, programé dos funciones separadas (`buscarLimiteInferior` y `buscarLimiteSuperior`) que siguen buscando hacia la izquierda o hacia la derecha aun después de encontrar el valor, para asegurarme de agarrar el primer y el último duplicado sin perder eficiencia.